MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Laboratory #5 – Range Sensor Introduction

Student Names: Brady Cabral and Emily Vargas-Penaranda
Instructor Name: Prof. Surgenor
Date Report Submitted: March 15, 2026

Summary:
The objective of this lab was to learn the calibration and implementation of an infrared range sensor, applied in the MiniBot mobile robot for obstacle detection by programming it to detect obstacles. The lab consisted of three main tasks. Firstly, the wiring was confirmed to be properly done, with the black wire to ground, red wire to battery (3V) rail and yellow wire (signal) to analog pin A3. For the second task, a calibration curve was generated by measuring the sensor output voltage at different distances, using both a digital mutimeter (DMM) and the ItsiBitsy microcontroller's analog-to-digital converter, yielding **Figure 1**. Then, the sensor output was plotted in real time using the Arduino serial plotter to observe the response and noise characteristics, shown in **Figure 2**. Finally, the MiniBot was programmed to stop within 20 cm of an obstacle placed when following a closed-loop black tape line, illustrated in **Figure 3**.

Hardware:
The hardware used in this lab consisted of the MiniBot robot equipped with the Sharp GP2D12 infrared range sensor mounted on the front of the robot. The sensor contains an infrared LED emitter and position-sensitive photodetector that measures reflected infrared light to determine distance. The sensor outputs an analog voltage that varies non-linearly, as shown in **Figure 1**.

Software:
A listing of the program ***Group30Lab5ObstacleDetect*** used in the lab is given in **Appendix A**. The program first initializes all the pins then runs a loop waiting for the red button to be pressed whilst flashing the green LED. Afterwards the program uses adapted code from lab 4 to read sensor data from line sensors and reads from the Sharp GP2D12. The program then maps the value to millivolts to be more understandable and checks if the value is less than the maximum permitted value (which is estimated to represent around 20cm) then the if the value is less than this maximum value, meaning the bot is further than 20cm from an obstacle the bot acts as it does in lab 4, following the black line. However, if the Sharp GP2D12 value is greater then bot stops the motors by using the *runMotors* function with the added delta set to zero for each motor thus motors get sent the stop pulse length and stop moving.

Results*:*
The sensor calibration plot is shown in **Figure 1**. The first measurement taken was at 90cm, resulting in a voltage of approximately 400mV, indicating the sensor was functioning as expected. As the target moved closer to the sensor, the output voltage increased in a non-linear fashion. Voltage peaked at a distance of roughly 7cm and started decreasing thereafter. Using the Arduino serial plotter, the sensor output was observed while moving the target in a sequence of 20 cm, 10cm, 20cm, 30 cm and back to 20 cm, shown in **Figure 2** The plot shows small fluctuations and noise, as expected from analog sensors.

Q1. The recommended supply voltage for the Sharp GP2D12 ranges from 4.5-5.5 V. This implies that the sensor must be supplied with a stable voltage in this range to operate accordingly. If the supply voltage varies significantly, the output voltage may shift and the resulting calibration curve will change, leading to incorrect distance approximation for the MiniBot.

Q2. The voltage output at a distance of 10cm is approximately 2.6V, and approximately 0.4V at a distance of 80cm. These values can vary as the sensor output varies slightly depending on the reflectivity of the target surface [1].
Q3. The sensor's approximate response time is 39 milliseconds. This means that the sensor produces a measurement roughly every 40 milliseconds, implying that the robot will not be able to detect changes in distance faster than the response time. This limitation must be considered in the robot's applications [1].
Q4. Yes, the target surface's reflectance can significantly affect sensor output, as the surface color, texture, and reflectivity can alter the intensity of the reflected infrared light; possibly leading to distance inaccuracies. For instance, dark-colored surfaces return a weaker infrared light, causing the sensor to underestimate the surface's distance. Conversely, surfaces with high reflectivity like mirros can cause the sensor to behave erratically, often returning unpredictable voltage values. Ideally, white and grey matte surfaces return the most stable sensor output results [2].

1. https://engineering.purdue.edu/ME588/SpecSheets/sharp_gp2d12.pdf
2. https://zbotic.in/sharp-infrared-distance-sensors-gp2d12-series-guide/?srsltid=AfmBOorWci295RATwgxUxKnbupmLt0MiOPgILSPbzq8AFUFuZc1g8fHE

Conclusions:
In this lab, the Sharp GP2D12 Infrared Sensor was calibrated and incorporated into the MiniBot for an obstacle-detection application. The calibration process showed that the relationship between output voltage and target distance is non-linear, peaking at a certain distance and reducing onwards. This showed the importance of understanding the behavior of employed tools, as the values obtained from the graph were used to program the robot to stop at a distance of 20cm from the target.

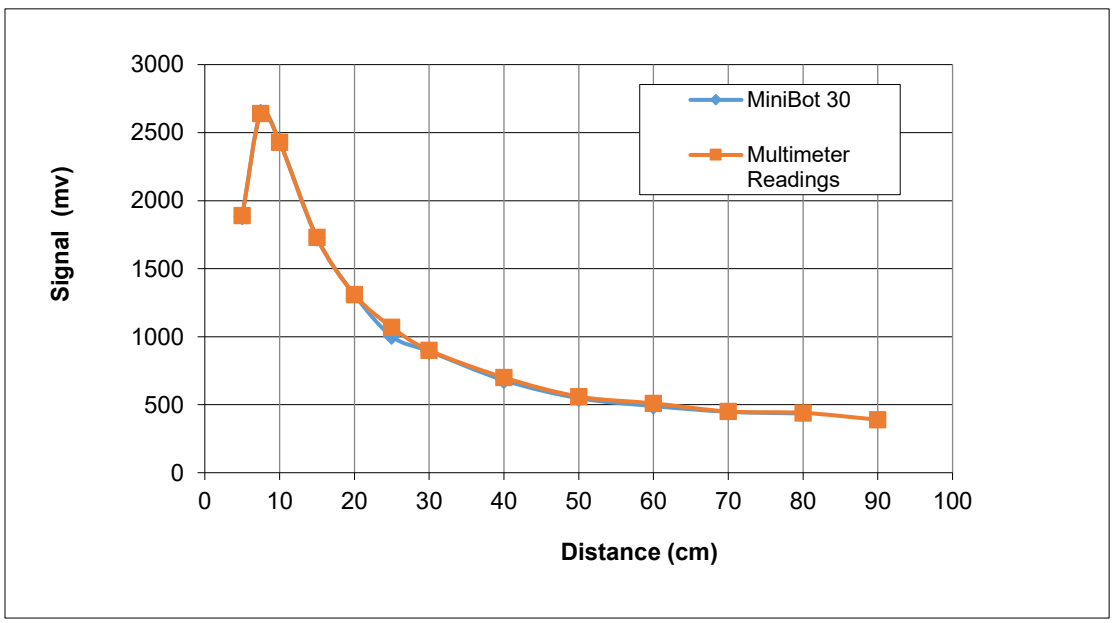

Fig 1. Signal strength versus distance results using DMM and Minibot serial output

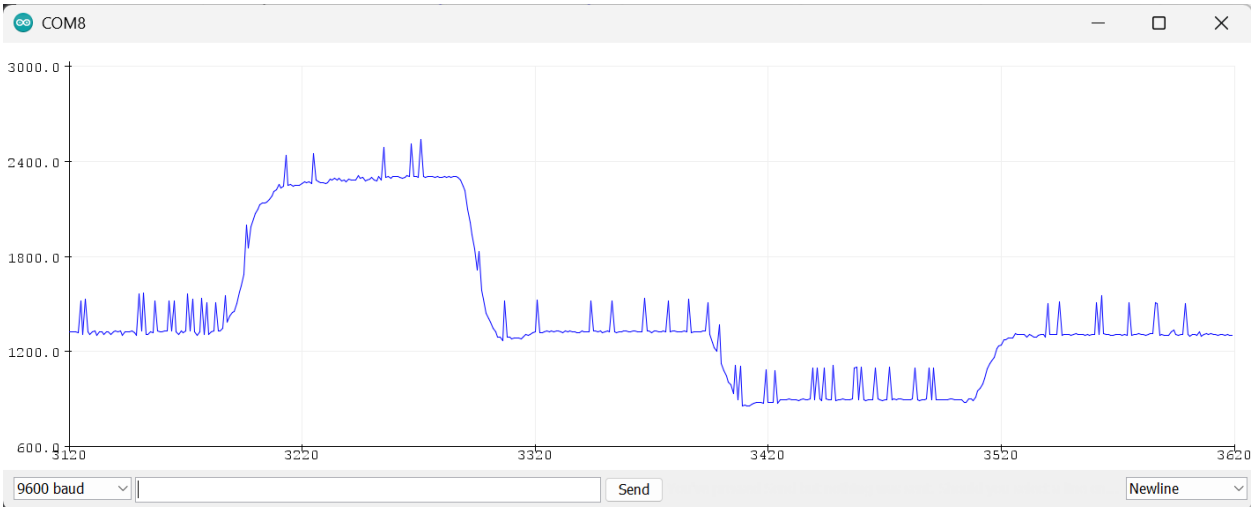

Fig. 2. Figure generated from Minibot based on range sensor data

Fig 3. MiniBot on track testing

Appendix A - Program Listing:

```
/**************************************************
********
  Group30Lab5ObstacleDetect

  Original by H. Fernando, 23/09/2021
   Updated by Brady Cabral & Emily Vargas-Penaranda
(15/03/2026)

  Used to follow a black line and stop when a wall is detected
**************************************************
********/

#define blcktp 2000
#define DELTA 10
#define MAX_DIST 1250

#include <Servo.h>
Servo leftWheel;
Servo rightWheel;


// Pin Assignments
int RED = 10;          //red LED Pin
int GRN = 9;          //green LED Pin
int YLW = 5;          //yellow LED Pin
int BUTTON = 7;       //pushbutton Pin
int SHARP = A3;     // Sharp Sensor on Analog Pin 3
int MOTOR_R = 3;       // right motor signal pin
int MOTOR_L = 4;       // left motor signal pin

boolean loopbttn = true;

const int LSENSOR = A2; // Left Sensor on Analog Pin 1
const int RSENSOR = A1; // Right Sensor on Analog Pin 2

const int stopPulse = 149;  // stop speed for motors (default =
155)

const boolean PLOT = true;  //true = plot; false = print

int value = 0;
int mv_value = 0;

//global variables
int lvalue = 0;  //left sensor value
int rvalue = 0;  //right sensor value

// Set-up Routine
void setup() {

// Initialize led pins as outputs.
  pinMode(GRN, OUTPUT);
  pinMode(YLW, OUTPUT);
  pinMode(RED, OUTPUT);

// Initialize pins as inputs
  pinMode(BUTTON, INPUT);
  pinMode(SHARP, INPUT);

  pinMode(LSENSOR, INPUT);
  pinMode(RSENSOR, INPUT);

  leftWheel.attach(MOTOR_L);
  rightWheel.attach(MOTOR_R);

// Initialize serial printout
  Serial.begin(9600);    // default 9600

  runMotors(0,0);
}

// Main Routine
void loop() {

    while (loopbttn && digitalRead(BUTTON) == LOW) {
      digitalWrite(GRN, HIGH);
      delay(125);
      digitalWrite(GRN, LOW);
      delay(125);
    }
    loopbttn = false;

    lvalue = analogRead(LSENSOR);
    rvalue = analogRead(RSENSOR);

    lvalue = map(lvalue,0,1023,0,3000);
    rvalue = map(rvalue,0,1023,0,3000);

    value = analogRead(SHARP);
      mv_value = map(value,0,1023,0,3300);  //convert AtoD
count to millivolts


    if (mv_value < MAX_DIST) {
      if (rvalue > blcktp && lvalue > blcktp) {
      turnOnLED(GRN);
      runMotors(DELTA, DELTA);
    }
    // Left side
    else if (rvalue > blcktp){
      turnOnLED(YLW);
      runMotors(DELTA, 0);

    }/* Right side */else if (lvalue > blcktp) {
      turnOnLED(RED);
      runMotors(0, DELTA);
    }/* Straddle */ else if (rvalue < blcktp && lvalue < blcktp){

      turnOnLED(GRN);
      runMotors(DELTA,DELTA);
    }
    } else runMotors(0,0);



}

//********** Functions (subroutines) *****************

// run robot wheels
void runMotors(int deltaL, int deltaR)
{
  int pulseL = (stopPulse + deltaL)*10;    //length of pulse in
microseconds
  int pulseR = (stopPulse + deltaR)*10;
  leftWheel.writeMicroseconds(pulseL);
  rightWheel.writeMicroseconds(pulseR);
}

// Turn on a single LED and turn others off
void turnOnLED(int COLOUR)
{
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(COLOUR, HIGH);
}
```